MODEL TUNING & EVALUATION · POSTER 1

The Evaluation Workflow

From "what does it predict?" to "can I trust it?"

Evaluation is not one number at the end — it is five questions asked in order. Skip one and you can ship a model that looks brilliant and fails in production.

THE WHOLE JOB IN ONE LINE

"Would this model still be right on a slab it has never seen before?"

THE FIVE-STAGE WORKFLOW

STAGE 1
Predict
What does the model actually estimate? The conditional mean.

→

STAGE 2
Validate
Does it hold on unseen data? Holdout & cross-validation.

→

STAGE 3
Tune
Improve it without cheating on the test set. Bias vs variance.

→

STAGE 4
Measure
Which metric matches the problem and the cost of errors?

→

STAGE 5
Diagnose
Do the assumptions hold? Residuals, collinearity, stability.

The one question behind it all

Every stage is a way of answering one thing honestly: **how well does this model do on data it has never seen?** Training performance is a diagnostic, not an achievement.

Fit ≠ Generalization **Accuracy ≠ Validity**
Software ran ≠ Answer is right

Three verdicts you're trying to avoid

GOLDEN RULES OF HONEST EVALUATION

1. **Judge on data the model never saw.** The cross-validation score is the truth; the training score is just a diagnostic.
2. **Match the metric to the decision,** not to habit. The cost of each error picks the metric.
3. **Keep one test set untouched** until the very end — tune on cross-validation only.
4. **A model can predict well and still break assumptions** — read metrics *and* diagnostics.
5. **Prevent group leakage:** the same slab in train and test inflates every score.
6. **Never trust a single number** — read fit and generalization together, always.

★ KEY TAKEAWAY Evaluation is a **sequence of honest questions**, not a final score. Ask all five, in order, every time.

UpSkill Sprint Consulting
upskillsprint.com

MODEL TUNING & EVALUATION · POSTER 2

The Metric Cheat Sheet

Match the metric to the problem — and to the mistake that costs you money.

The first rule of metrics: the metric must match the problem type, what the model predicts, and which error is actually expensive. Pick before you model, not after.

DECISION RULE

Let the cost of the error — a missed defect vs. a false alarm — choose the metric.

REGRESSION · PREDICTING A NUMBER

METRIC	MEASURES	FORMULA	REACH FOR IT WHEN
MAE	Average error size, all errors equal	`mean|y−ŷ|`	You want a plain, robust "typical miss."
MSE	Squared error — punishes big misses	`mean(y−ŷ)²`	Large errors are especially bad.

RMSE	MSE back in the target's own units	$\sqrt{MSE}$	You want MSE's penalty, readable units.
R^2	Share of variance explained (0→1)	$1-SS_r/SS_t$	Communicating fit to non-specialists.

CLASSIFICATION · A CLASS

Accuracy
(TP+TN)/all · balanced only

Precision
TP/(TP+FP) · false alarms costly

Recall
TP/(TP+FN) · a miss is dangerous

F1
balance, imbalanced data

ROC-AUC
separability across all thresholds; threshold-free comparison

The confusion matrix

	Pred +	Pred −
Act +	**TP**	**FN** missed
Act −	**FP** false alarm	**TN**

Every classification metric is just a ratio of these four counts.

Clustering · no labels

Silhouette — cohesion vs. separation (−1→+1). No ground truth; judge how tight the groups are.

Adjusted Rand Index — agreement with known labels, corrected for chance.

Quantile · a percentile

Pinball loss — asymmetric error at quantile τ (τ=0.5 → MAE; τ=0.9 penalizes under-prediction 9×).

Pseudo R^2 — fit vs. a null model. McFadden / Koenker–Machado form.

R^2 and RMSE measure the wrong target here — they assume you predicted the mean.

The costed-error rule

- **Missing a defect is dangerous** → optimize **Recall**
- **False alarms waste money** → optimize **Precision**
- **Big misses ruin you** → prefer **RMSE/MSE**
- **Imbalanced classes** → never trust **Accuracy**

KEY TAKEAWAY The metric must match the **problem type** and the **cost of the mistake**. Choose it before you model.

UpSkill Sprint Consulting
upskillsprint.com

MODEL TUNING & EVALUATION · POSTER 3

Overfitting & Validation Decoder

HEADLINE NUMBER
Cross-val R^2
The only score that estimates performance on brand-new data.

Three numbers tell you whether a model learned the signal or memorized the noise.

Read the three R^2 values together — never one alone. The gap between them, and the spread across splits, is the whole story of generalization.

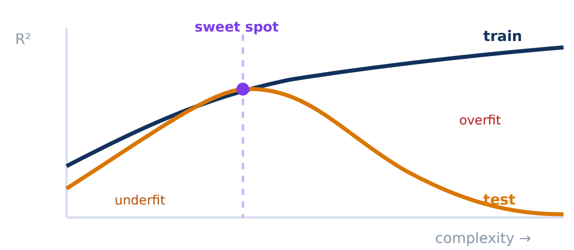
THE THREE R^2 VALUES, IN ORDER OF TRUST

① Training R^2
Fit on data it **has seen.** Diagnostic only — a high value alone proves nothing.

② Holdout R^2
Score on a **20% slice** held back. Your first honest generalization check.

③ Cross-val R^2
Averaged over **many splits** (± spread). **This is the headline number.**

THE COMPLEXITY CURVE — WHERE OVERFITTING LIVES



Training R^2 only ever climbs as you add complexity. **Test R^2** rises, peaks at the sweet spot, then **collapses** as the model starts memorizing noise.

Tuning is the hunt for that peak — the most complexity you can afford before the test score turns down.

Healthy model

Training R^2	**0.78**
Holdout R^2	**0.74**
CV R^2	**0.72 ± 0.03**

Training slightly higher; all three close; small spread.

Overfit model

Training R^2	**0.96**
Holdout R^2	**0.42**
CV R^2	**0.38 ± 0.10**

Huge train→test drop; large, unstable spread.

Bias vs variance

High bias / underfit: low everywhere. Too simple — add signal.

High variance / overfit: great train, poor test. Too complex — simplify or regularize.

Sweet spot: lowest *total* error on unseen data.

Grouped data → GroupKFold

If the same **slab** appears in train and test, the score is a lie. Group by `SlabID` so whole slabs stay together — then CV answers: *"can it predict a **brand-new** slab?"*

★ **KEY TAKEAWAY** Read the three R^2s **together.** A big train→test gap is overfitting; low-everywhere is underfitting.

MODEL TUNING & EVALUATION · POSTER 4

Regression Diagnostics Decoder

The block of numbers under a regression summary is an assumptions report card.

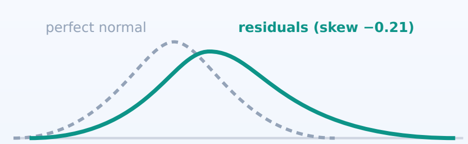
Diagnostics don't measure accuracy — they tell you whether you're even allowed to trust the coefficients. Here is each test, its benchmark, and the verdict from a real steel-process energy model.

THE #1 MISREAD
On big data, normality p-values almost always "fail." Trust skew & kurtosis, not the p-value.

THE SUMMARY BLOCK, DECODED

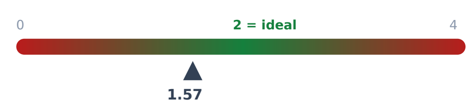
DIAGNOSTIC	TESTS FOR	BENCHMARK	STEEL VALUE	VERDICT
Omnibus p	Residual normality (skew+kurtosis)	> 0.05	0.000	Ignore — large N
Jarque-Bera p	Residual normality	> 0.05	~0	Ignore — large N
Skew	Symmetry of residuals	≈ 0	−0.21	Good
Kurtosis	Tail heaviness / outliers	≈ 3	2.61	Good
Durbin-Watson	Autocorrelation of residuals	≈ 2	1.57	Mild
Condition No.	Multicollinearity / stability	< 30	7,500	Act on it
AIC / BIC	Fit vs complexity trade-off	Lower (relative)	~917k	Compare only
VIF	Collinearity per variable	< 5 ok, >10 bad	per-term	Check each

Trust skew & kurtosis, not the p-value



Skew ≈ 0 and kurtosis ≈ 3 say the residuals are fine — even when Omnibus/JB "fail" on 60,000 rows.

Durbin-Watson = 1.57



Below 2 → mild positive autocorrelation. Check ordering/grouping; consider robust SEs.

The large-N trap

With tens of thousands of rows, normality tests flag microscopic, harmless departures (p ≈ 0). It sinks a lot of good analyses.

Fix multicollinearity

A high condition number / VIF makes coefficients unstable and their signs unreliable. **Drop or combine** correlated predictors, Judge normality by **skew, kurtosis, and a residual plot** — not the p-value.

standardize inputs, or use **ridge regression** — then interpret.

★ **KEY TAKEAWAY** Diagnostics test **trustworthiness, not accuracy.** Beware the large-N trap; act on condition number & DW.